Faisal Abdullah Alshehri

University ID: 2240001737

Lab 7(A) - Laplacian Filter

In [1]:
```
import cv2
import numpy as np

from IPython.display import Image, display
from matplotlib import pyplot as plt

plt.rc('text', usetex=False)
```

Apply Laplacian Filter on Image to get Sharp Image

In [2]:
```
# ================ ( Laplacian Filter ) ===================


# 0. Image file path
imgFile = "moon.jpg"

# 1. Load image and convert to grayscale
img = cv2.imread(imgFile, 0)

# 2. Manually define the Laplacian Kernel
# This is the standard 3x3 kernel that detects horizontal and vertical ed
manual_kernel = np.array([
    [0,  1,  0],
    [1, -4,  1],
    [0,  1,  0]
], dtype=np.float32)

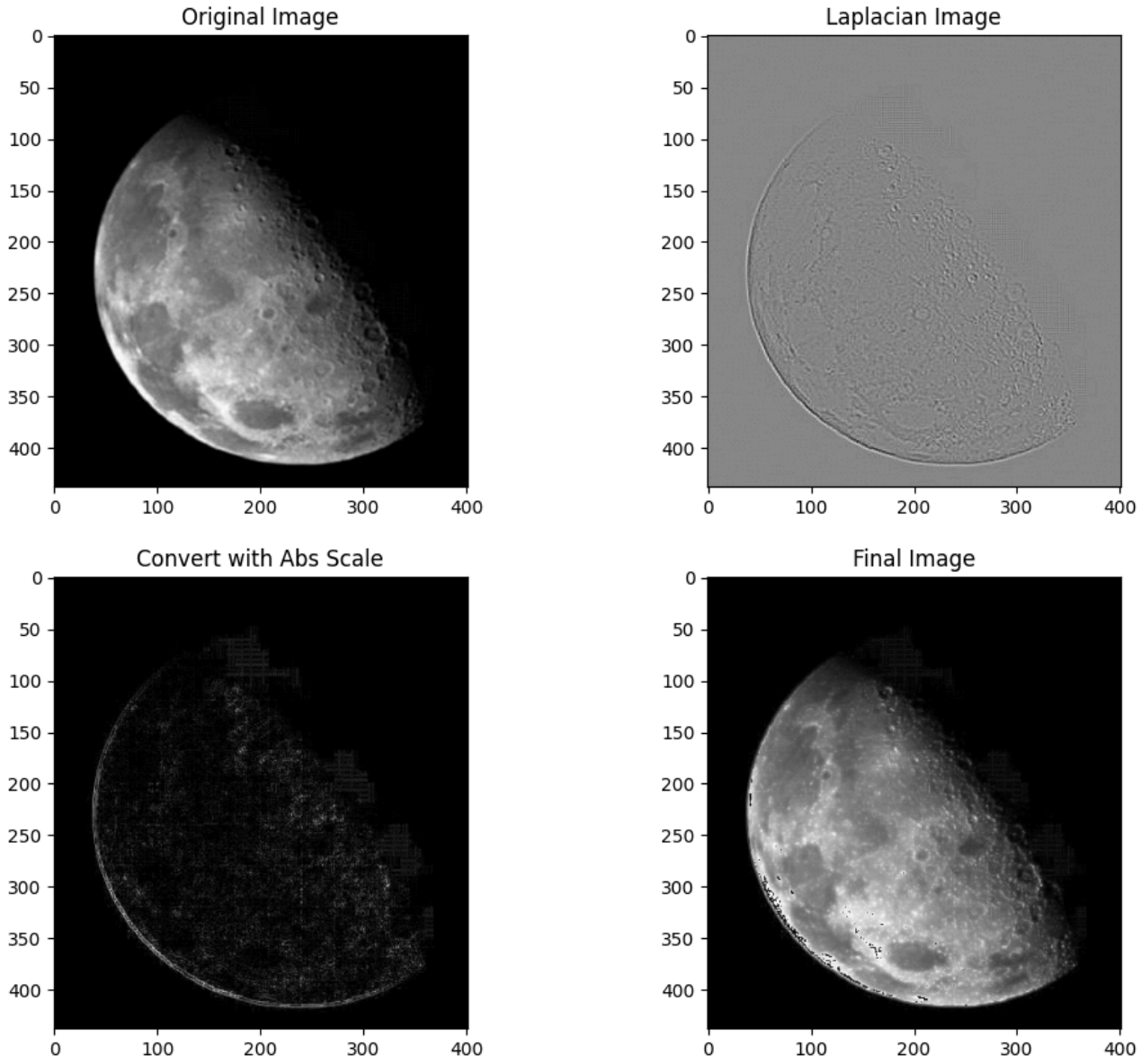
# Alternative kernel: Detects diagonal edges as well
# manual_kernel = np.array([
#     [1,  1,  1],
#     [1, -8,  1],
#     [1,  1,  1]
# ], dtype=np.float32)

# 3. Apply the filter using filter2D
# -1 indicates the output image will have the same depth as the source
# However, for Laplacian, use cv2.CV_64F to avoid clipping negative value
laplacian_manual = cv2.filter2D(img, cv2.CV_64F, manual_kernel)
```

```
# 4. Take absolute value and convert back to 8-bit (0-255)
laplacian_abs = cv2.convertScaleAbs(laplacian_manual)

finalImage = img + laplacian_abs

# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image
plt.subplot(222), plt.imshow(laplacian_manual, cmap='gray'), plt.title('L
plt.subplot(223), plt.imshow(laplacian_abs, cmap='gray'), plt.title('Conv
plt.subplot(224), plt.imshow(finalImage, cmap='gray'), plt.title('Final I
plt.show()
```



In [3]:
```
# ================== ( Laplacian Filter ) ===============

# 1. Optional: Apply Gaussian Blur to reduce noise
```

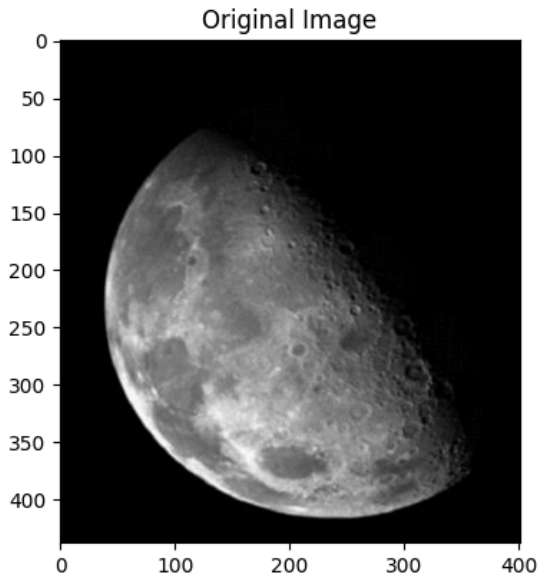
```
# This prevents the algorithm from picking up tiny details as edges
blurred = cv2.GaussianBlur(img, (3, 3), 0)

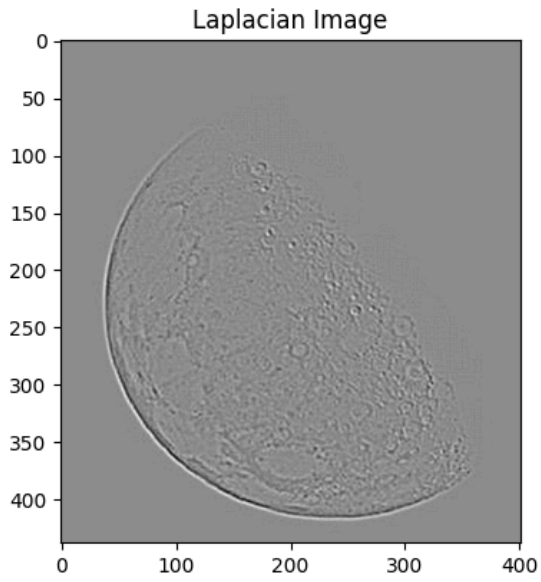
# 2. Apply the Laplacian Algorithm
# Use CV_64F to capture both positive and negative gradients (essential f
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)

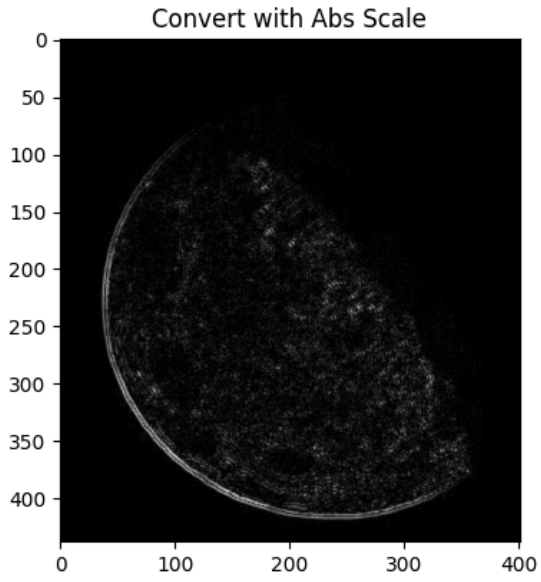
# 3. Convert back to 8-bit image to view it properly
laplacian_8bit = cv2.convertScaleAbs(laplacian)

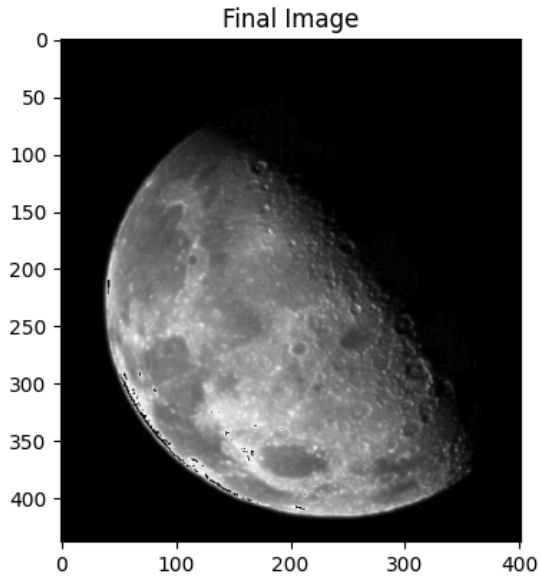
# 4. Adding Laplace and origianl Image
finalImage2 = img + laplacian_8bit

# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image
plt.subplot(222), plt.imshow(laplacian, cmap='gray'), plt.title('Laplacia
plt.subplot(223), plt.imshow(laplacian_8bit, cmap='gray'), plt.title('Con
plt.subplot(224), plt.imshow(finalImage2, cmap='gray'), plt.title('Final
plt.show()
```









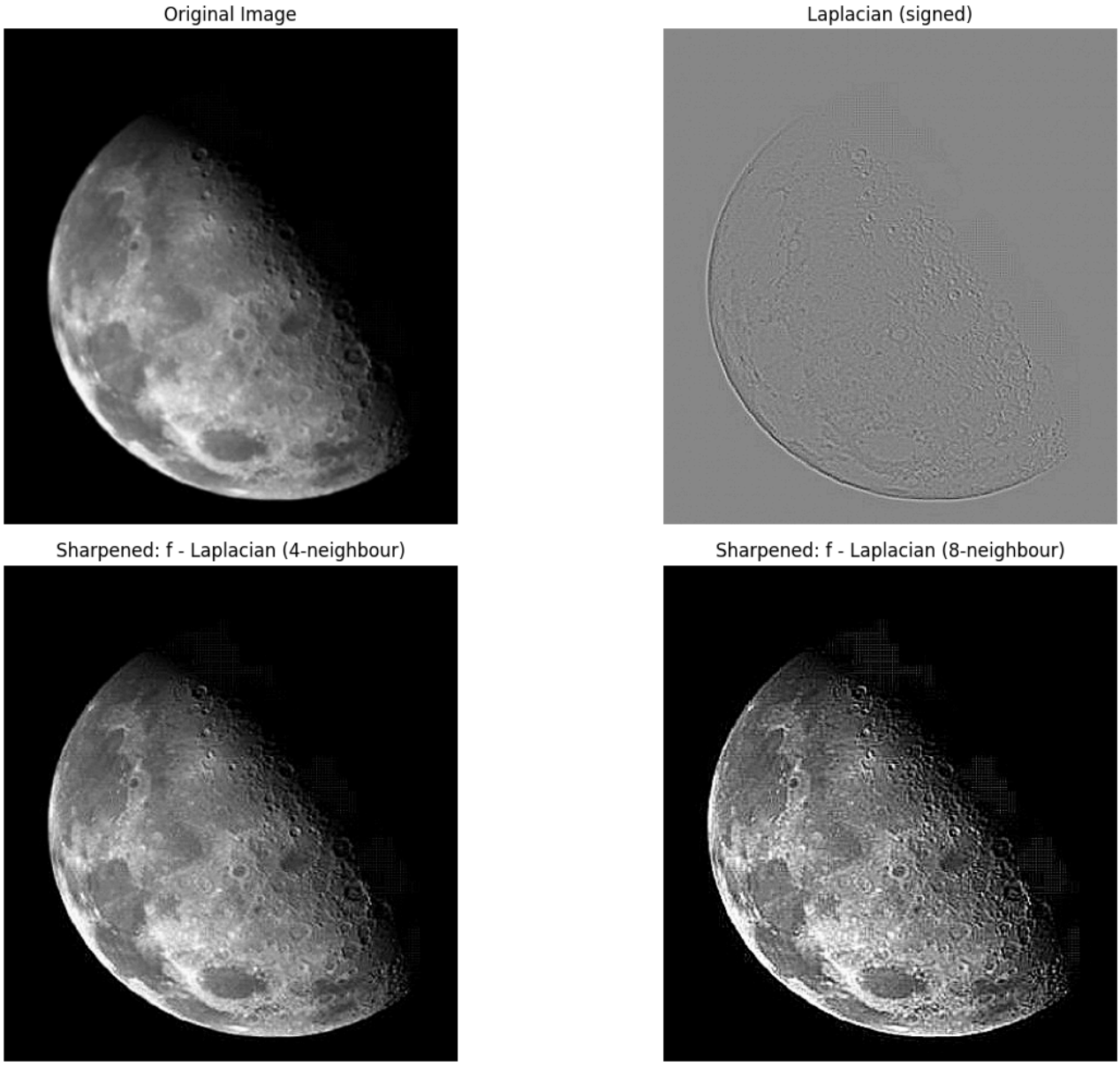
Corrected Sharpening (Laplacian)

In the two cells above, `img + laplacian_abs` is a `uint8 + uint8` addition, so values above 255 **wrap around** (e.g. 250 + 20 = 14), and `abs()` throws away the sign of the edge. With a kernel whose centre is negative (−4), the correct sharpening is `g = f − ∇²f`, computed in a signed type and then clipped to 0–255.

In [4]:
```
# Corrected sharpening: g = f - Laplacian(f), computed in float64 then cl
img_f = img.astype(np.float64)
lap = cv2.filter2D(img_f, cv2.CV_64F, manual_kernel)          # signed La
sharp = np.clip(img_f - lap, 0, 255).astype(np.uint8)          # center i
```

```
# Same result using the diagonal-aware kernel (8-neighbour)
kernel8 = np.array([[1, 1, 1],
                    [1,-8, 1],
                    [1, 1, 1]], dtype=np.float64)
sharp8 = np.clip(img_f - cv2.filter2D(img_f, cv2.CV_64F, kernel8), 0, 255

plt.figure(figsize=(14, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image
plt.subplot(222), plt.imshow(lap, cmap='gray'), plt.title('Laplacian (sig
plt.subplot(223), plt.imshow(sharp, cmap='gray'), plt.title('Sharpened: f
plt.subplot(224), plt.imshow(sharp8, cmap='gray'), plt.title('Sharpened:
for i in range(1, 5): plt.subplot(220 + i).axis('off')
plt.tight_layout()
plt.show()
```



Observation: the Laplacian highlights the crater edges and fine surface

detail of the moon (high-frequency regions) and gives zero in smooth areas. Subtracting it from the original boosts local contrast, so craters look sharper. The 8-neighbour kernel responds to diagonal edges as well, so it sharpens slightly more.